Practical 1: Student Marks Analyzer

Problem Definition

Store student data as a list of tuples. Remove duplicate marks, calculate average marks and store result in a dictionary.

Sample Input

```
students = [
    (1, "Amit", [78, 85, 90, 85]),
    (2, "Neha", [88, 92, 79, 88]),
    (3, "Ravi", [70, 75, 80, 75])
]
```

Sample Output

```
{1: ('Amit', 84.33), 2: ('Neha', 86.33), 3: ('Ravi', 75.0)}
```

In [21]:
```
students = [
    (1, "Amit", [78, 85, 90, 85]),
    (2, "Neha", [88, 92, 79, 88]),
    (3, "Ravi", [70, 75, 80, 75])
]

d1 = {}
for i, j, k in students:
    l1 = sum(set(k)) / len(set(k))
    t1 = (j, l1)
    d1[i] = t1
print(d1)
```

```
{1: ('Amit', 84.33333333333333), 2: ('Neha', 86.33333333333333), 3: ('Ravi', 75.
0)}
```

Practical 2: Inventory Management

Problem Definition

Remove out-of-stock items and calculate total inventory value.

Sample Input

```
inventory = {
    'pen': (10, 50),
    'book': (100, 0),
    'pencil': (5, 100)
}
```

Sample Output

```
    {'pen': (10, 50), 'pencil': (5, 100)}
    Total Value: 1000
```

In [59]:
```python
inventory = {
    'pen': (10, 50),
    'book': (100, 0),
    'pencil': (5, 100)
}
d1 = {}
total_val = 0
for k, v in inventory.items():
    if v[1] == 0:
        continue
    d1[k] = v
    total_val += v[0] * v[1]
print(d1)
print("Total Value :", total_val)
```

```
{'pen': (10, 50), 'pencil': (5, 100)}
Total Value : 1000
```

Practical 3: Email Domain Extractor

Problem Definition

Extract email domains and count occurrences.

Sample Input

```python
emails = ['a@gmail.com', 'b@yahoo.com', 'c@gmail.com']
```

Sample Output

```
    {'gmail.com': 2, 'yahoo.com': 1}
```

In [72]:
```python
emails = ['a@gmail.com', 'b@yahoo.com', 'c@gmail.com']
d1 = {}
l1 = []
for i in emails:
    l1.append(i.split("@")[1])
for j in l1:
    if j in d1:
        d1[j] += 1
    else:
        d1[j] = 1
print(d1)
```

```
{'gmail.com': 2, 'yahoo.com': 1}
```

Practical 4: Shopping Cart Billing

Problem Definition

Calculate item-wise cost and total bill.

Sample Input

```
cart = [('apple',2), ('banana',3)]
prices = {'apple':50, 'banana':10}
```

Sample Output

```
{'apple': 100, 'banana': 30}
Total Bill: 130
```

In [121…

```
cart = [('apple', 2), ('banana', 3)]
prices = {'apple': 50, 'banana': 10}
d1 = {}
total_bill = 0

for item, qut in cart:
    cost = qut * prices[item]
    d1[item] = cost
    total_bill += cost

print(d1)
print("Total Bill: ", total_bill)
```

```
{'apple': 100, 'banana': 30}
Total Bill:  130
```

Practical 5: Character Position Index

Problem Definition

Store positions of each character in a string.

Sample Input

```
s = "banana"
```

Sample Output

```
{'b': [0], 'a': [1, 3, 5], 'n': [2, 4]}
```

In [130…

```
s = "banana"
pos = {}
for idx, char in enumerate(s):
    if char in pos:
        pos[char].append(idx)
    else:
        pos[char] = [idx]
print(pos)
```

```
{'b': [0], 'a': [1, 3, 5], 'n': [2, 4]}
```

Practical 6: Survey Data Aggregation

Problem Definition

Count responses city-wise and choice-wise.

Sample Input

```
responses = [
    ('Amit','Rajkot','Yes'),
    ('Neha','Rajkot','No'),
    ('Ravi','Ahmedabad','Yes')
]
```

Sample Output

```
{'Rajkot': {'Yes': 1, 'No': 1}, 'Ahmedabad': {'Yes': 1}}
```

In [146…

```
responses = [
    ('Amit','Rajkot','Yes'),
    ('Neha','Rajkot','No'),
    ('Ravi','Ahmedabad','Yes')
]
ans = {}
for name, city, choice in responses:
    if city not in ans:
        ans[city] = {}
    if choice not in ans[city]:
        ans[city][choice] = 0
    ans[city][choice] += 1
print(ans)
```

```
{'Rajkot': {'Yes': 1, 'No': 1}, 'Ahmedabad': {'Yes': 1}}
```

In [ ]: